TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

Module 5: Distributed Data Parallelism

Replicas, Collectives, and Global Invariants

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. The data-parallel contract
2. Overlap and rank-aware state
3. Correctness before scaling

Guiding question

When does training on W GPUs reproduce one larger-batch update, and where can that equivalence fail?

What should be clear by the end?

1. Derive the averaged gradient computed by data parallelism.
2. Reason about ring all-reduce cost and overlap.
3. Keep accumulation, sampling, logging, and checkpointing rank-aware.
4. Measure both numerical equivalence and scaling efficiency.

PART 1

The data-parallel contract

1. The data-parallel contract
2. Overlap and rank-aware state
3. Correctness before scaling

Each rank owns a full model and different examples

- one process controls one GPU
- parameters begin identical on all ranks
- local batches are disjoint parts of a global batch
- gradients synchronize before every optimizer update

Global batch

$$B_{\text{global}} = W\, B_{\text{local}}, K$$

K is the number of accumulated micro-steps.

Gradient averaging recovers the global mean

Rank r computes a mean gradient over n_r valid tokens:

$$g_r = \frac{1}{n_r} \sum_{i \in \mathcal{B}_r} \nabla_\theta \ell_i$$

If every $n_r = n$, DDP's average is:

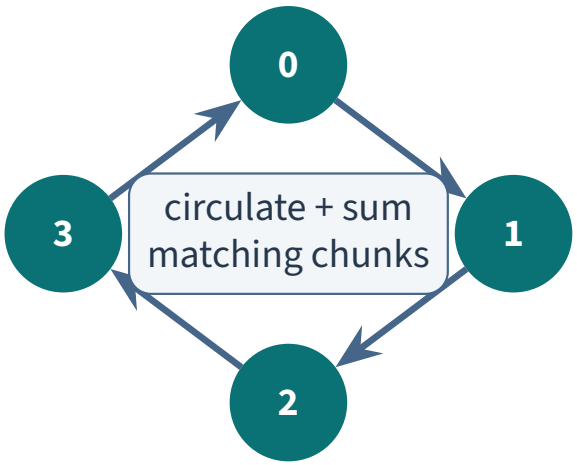
$$\bar{g} = \frac{1}{W} \sum_{r=0}^{W-1} g_r = \frac{1}{Wn} \sum_{r,i} \nabla_\theta \ell_i$$

Unequal valid-token counts

Averaging rank means is not the global token mean. Aggregate summed loss and counts, or weight gradients correctly.

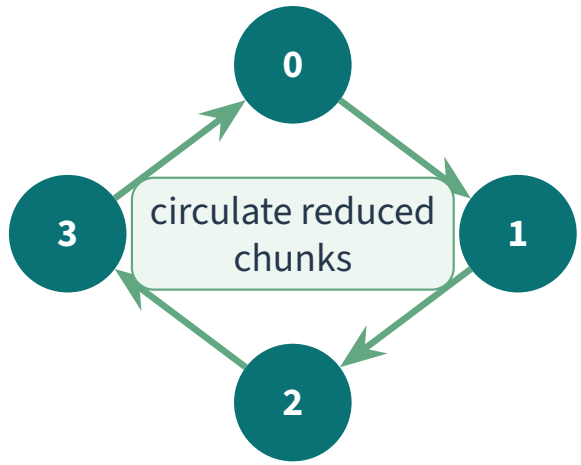
A ring all-reduce distributes the traffic

1. reduce-scatter



one reduced chunk per rank

2. all-gather



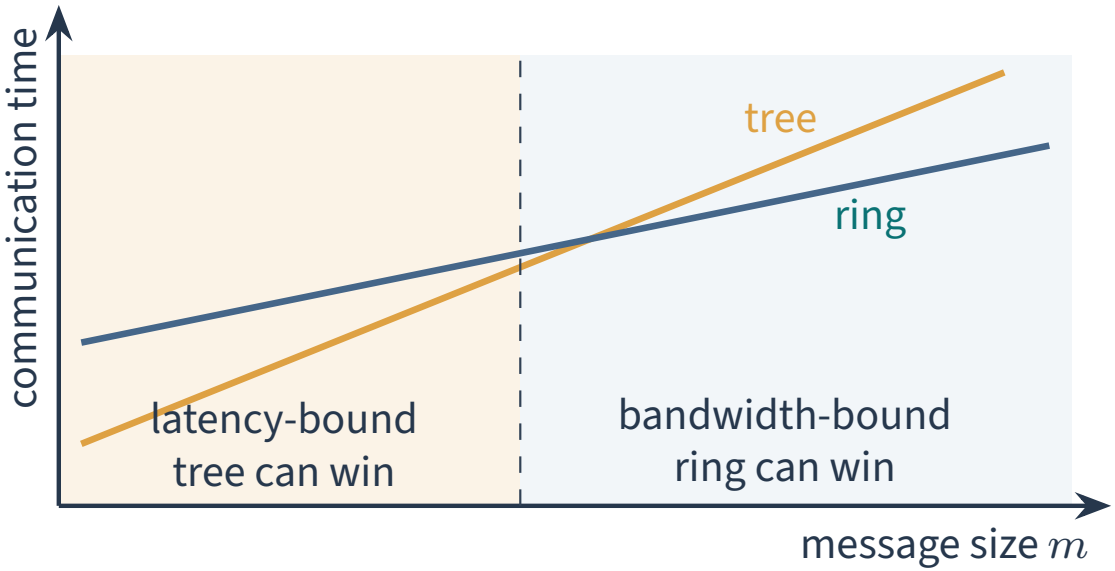
full reduced buffer on every rank

Per-rank payload for gradient buffer S

$2\frac{W-1}{W}S$ bytes, ignoring latency and protocol overhead.

Source: Hugging Face Ultra-Scale Playbook

Collective latency and bandwidth crossover



$$T_{\text{comm}}(m) \approx \alpha + \beta m$$

α
fixed launch latency

β
time per byte, roughly $1/BW$

Source: MVA Training and Deploying Large-Scale Models, Lecture 2 (2026)

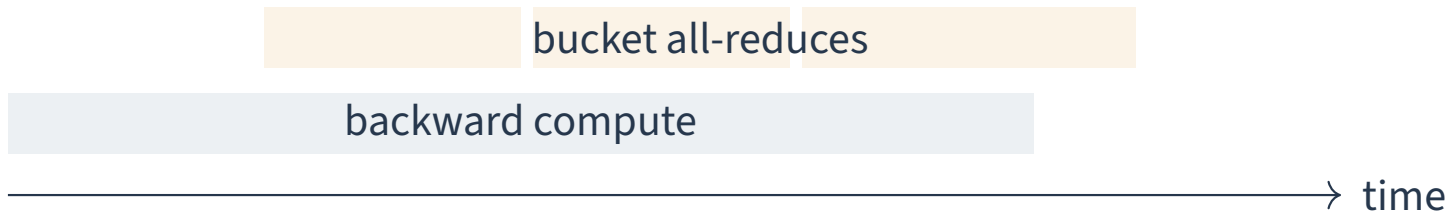
PART 2

Overlap and rank-aware state

1. The data-parallel contract
2. Overlap and rank-aware state
3. Correctness before scaling

Buckets overlap communication with backward

Backward produces gradients from later layers first. DDP groups parameters into buckets and launches a collective as each bucket becomes ready.



Tail exposure

Communication that remains after backward completes is visible step-time overhead.

Accumulation should synchronize once per update

- Early micro-steps accumulate locally under `no_sync()`.
- The final micro-step triggers the bucket all-reduces.
- Loss normalization still targets the global mean.

Common waste

Synchronizing every micro-step communicates K times for one optimizer update.

Common bug

Never synchronizing lets replicas drift immediately after the update.

Distributed sampling is part of correctness

- Each rank needs a disjoint subset within an epoch or sampling interval.
- All ranks must agree on the shuffle seed and epoch counter.
- Dropping or padding the tail changes sample multiplicity.
- Resume must restore the sampler’s logical position.

Fast audit

Log sample IDs per rank for a few updates. Check within-step overlap, missing examples, and resume continuity.

Only some side effects should happen on every rank

Action	Every rank?	Reason
forward and backward	yes	local data contribution
optimizer update	yes	replicas remain identical
metric numerator/count	yes	later reduction
console and tracker logging	usually rank 0	avoid duplicates
full checkpoint publication	coordinated	avoid corruption
validation sampling	sharded or rank 0	declare aggregation

Checkpoint: batch accounting

Think, then compare

With $W = 8$, micro-batch 2, context 2048, and four accumulated micro-steps, compute tokens per update. How many gradient synchronizations should occur?

PART 3

Correctness before scaling

1. The data-parallel contract
2. Overlap and rank-aware state
3. Correctness before scaling

A five-rung correctness ladder

1. parameters are identical immediately after initialization
2. one global-batch update matches a single-process reference
3. parameters remain synchronized after the update
4. sample IDs show the intended global data coverage
5. save and resume preserve model, optimizer, RNG, and sampler state

Tolerance is part of the test

Reduction order changes floating-point rounding. Define acceptable numerical error rather than demanding bitwise equality by accident.

Scaling efficiency separates speed from more work

$$\text{speedup}(W) = \frac{t_1}{t_W}, \qquad \text{efficiency}(W) = \frac{\text{speedup}(W)}{W}$$

Strong scaling

Hold the global batch and work fixed. Ask how much faster the same update becomes.

Weak scaling

Hold work per GPU fixed. Global batch grows with W.

A trace explains poor scaling

- long exposed collectives suggest insufficient compute overlap or slow links
- rank skew suggests data loading, imbalance, or stragglers
- many small collectives suggest poor bucketing
- gaps before collectives suggest autograd readiness or synchronization barriers
- identical time with more GPUs may mean the local batch became too small

Checkpoint: equivalence failure

Think, then compare

A two-GPU run has the expected throughput, but its first update differs sharply from the single-process reference. Give a check order covering initialization, data, loss normalization, reduction, and optimizer state.

The durable ideas

1. DDP is equivalent to a global mean update only when data and normalization invariants hold.
2. Ring all-reduce spreads bandwidth cost, while buckets hide part of it behind backward.
3. Sampling, metrics, side effects, and resume logic must all understand rank.
4. Correctness comparisons come before speedup and efficiency measurements.